BASH SCRIPTING FUNDAMENTALS

Variables & Quoting

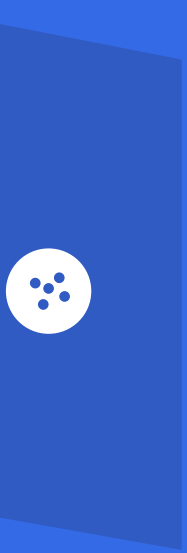
Store data safely and avoid word-splitting bugs

Merit Advisory

Bash Scripting Fundamentals Bootcamp

Beginner-friendly lecture materials

July 2026



WHAT WE WILL COVER

Agenda

1. Section 1: Variable Fundamentals — 4 slides
2. Section 2: Quoting Essentials — 4 slides
3. Section 3: Word Splitting and Globbing — 4 slides
4. Section 4: Defaults and Required Values — 4 slides
5. Section 5: String Parameter Expansion — 4 slides
6. Section 6: Declarations and Variable Lifetime — 4 slides
7. Section 7: Environment and PATH — 4 slides
8. Section 8: Positional Parameters and Options — 4 slides

9 Section 9: Reading Input and IFS — 4 slides

10 Section 10: Arrays and Arithmetic — 4 slides

11 Section 11: Substitution and Special Variables — 4 slides

12 Section 12: Formatting, Comparisons, and Review — 4 slides

1 Section 1

Variable Fundamentals

1.1 Variable Assignment · No Spaces Around Equals

- A Bash variable stores a text value under a name.
- Assignment uses name=value with no spaces around the equal sign.
- The value belongs to the current shell unless you export it.
- Spaces around the equal sign change the meaning of an assignment.
- Bash treats words separated by spaces as command names and arguments.
- Write assignments tightly so the shell recognizes them as variables.

Examples

```
# Assign a word
course=bash
# Assign a path
log_file=app.log
# Correct assignment
name=Sam
# Incorrect shape
name = Sam
```

1.2 Reading Variables · Variable Names

- A dollar sign reads the value stored in a variable.
- The variable name itself is not printed when you use dollar expansion.
- If the variable is unset, Bash usually expands it to an empty string.
- Bash variable names commonly use letters, numbers, and underscores.
- A variable name cannot start with a number.
- Clear names make scripts easier to read and maintain.

Examples

```
# Read a value
name=Sam
echo "$name"
# Use in text
course=bash
echo "Course: $course"
# Use a clear name
report_date=2026-07-28
# Use underscore
input_file=data.txt
```

1.3 Empty Variables · unset

- A variable can be set to an empty value.
- Empty and unset are different states, but both may print as nothing.
- Quoting helps you see and preserve empty values safely.
- The unset command removes a variable from the current shell.
- After unset, normal expansion produces an empty value unless strict checks are used.
- Unset is useful when a value should not be reused by later commands.

Examples

```
# Assign empty value
name=""
printf "[%s]\n" "$name"
# Unset then print
unset name
printf "[%s]\n" "$name"
# Unset a variable
token=abc
unset token
# Show after unset
unset token
printf "[%s]\n" "$token"
```

1.4 readonly - Protect a Variable

- The readonly command prevents a variable from being changed or unset.
- It is useful for constants that should stay fixed during a script.
- Trying to change a readonly variable causes an error.

Examples

```
# Create readonly value
readonly app_name="demo"
# Declare readonly
declare -r region="us-east"
```

2 Section 2

Quoting Essentials

2.1 Double Quotes · Single Quotes

- Double quotes keep a value together as one word after expansion.
- Variables and command substitutions still expand inside double quotes.
- Most variable expansions in scripts should be double-quoted.
- Single quotes preserve text exactly as written.
- Variables do not expand inside single quotes.
- Use single quotes when you want a dollar sign or other special character to stay literal.

Examples

```
# Preserve spaces
name="Ada Lovelace"
echo "$name"
# Expand in sentence
item=file
echo "Selected: $item"
# Print literal dollar
echo '$HOME'
# Print literal text
echo 'Use $name here'
```

2.2 Unquoted Variables · $var Versus Quoted Var

- An unquoted variable can be split into multiple words.
- It can also trigger filename matching when the value contains wildcard characters.
- Unquoted expansion is a common source of beginner bugs.
- The form $var expands the variable and then lets the shell process the result.
- The quoted form keeps the expanded value as one argument.
- Use the quoted form when the value comes from a user, file, or earlier command.

Examples

```
# Unsafe display
name="Ada Lovelace"
echo $name
# Safer display
name="Ada Lovelace"
echo "$name"
# One argument safely
file="my notes.txt"
ls "$file"
# Multiple words unsafely
file="my notes.txt"
ls $file
```

2.3 Braces Around Variable Names · Escaping Special Characters

- Braces mark exactly where a variable name begins and ends.
- They are helpful when text touches the variable name.
- The forms $name and ${name} read the same variable when no boundary is needed.
- A backslash can remove special meaning from the next character.
- Escaping is useful inside double quotes when you need a literal dollar sign.
- Too many escapes make scripts hard to read, so prefer clear quoting first.

Examples

```
# Add suffix safely
name=report
echo "${name}.txt"
# Avoid name confusion
day=Mon
echo "${day}day"
# Literal dollar
echo "Price is \$5"
# Literal quote
echo "She said \"hi\""
```

2.4 Nested Quoting

- Nested quoting appears when one command contains another command string.
- Choose the outer quotes so the inner command expands at the intended time.
- Keeping nested examples small makes quoting easier to reason about.

Examples

```
# Single outside
bash -c 'echo "$HOME"'
# Double outside
name=Sam
bash -c "echo hello"
```

3 Section 3

Word Splitting and Globbing

3.1 Word Splitting · Globbing After Expansion

- After unquoted expansion, Bash can split text into words.
- The default split characters are spaces, tabs, and newlines.
- Double quotes prevent this splitting for a variable value.
- Globbing is filename matching with patterns such as star and question mark.
- Unquoted variable values can become glob patterns after expansion.
- Quoting the variable keeps wildcard characters as literal text.

Examples

```
# Observe splitting
items="one two"
printf "<%s>\n" $items
# Prevent splitting
items="one two"
printf "<%s>\n" "$items"
# Unsafe pattern
pattern="*.txt"
printf "%s\n" $pattern
# Literal pattern
pattern="*.txt"
printf "%s\n" "$pattern"
```

3.2 Splitting Before Globbing · IFS Default Separators

- Bash performs word splitting before filename matching on unquoted results.
- A value with spaces and stars can turn into many separate arguments.
- This order explains why one missing pair of quotes can change a command dramatically.
- IFS is the variable Bash uses for some word splitting operations.
- By default, it includes space, tab, and newline characters.
- Changing IFS affects parsing, so keep changes narrow and intentional.

Examples

```
# Show risky value
value="logs *.txt"
printf "<%s>\n" $value
# Keep one value
value="logs *.txt"
printf "<%s>\n" "$value"
# Show default effect
list="a b c"
printf "%s\n" $list
# Set a comma separator
IFS=,
line="a,b,c"
# ...
```

3.3 Custom IFS for One Read · Disable Globbing Briefly

- A temporary IFS assignment can change how read separates fields.
- Putting IFS before read limits the assignment to that command.
- This pattern is safer than changing IFS for the whole script.
- The set -f command disables filename expansion in the current shell.
- The set +f command turns filename expansion back on.
- Quoting variables is usually clearer than disabling globbing globally.

Examples

```
# Split CSV fields
IFS=, read -r a b c <<< "red,green,blue"
# Print fields
printf "%s %s %s\n" "$a" "$b" "$c"
# Turn globbing off
set -f
# Turn globbing on
set +f
```

3.4 Safe Arguments Pattern

- Treat variable values as data unless you intentionally want shell syntax.
- Double-quote each variable when passing it as a command argument.
- This pattern protects spaces, empty values, and wildcard characters.

Examples

```
# Safe file argument
file="my report.txt"
cat "$file"
# Safe directory argument
dir="old logs"
mkdir -p "$dir"
```

4 Section 4

Defaults and Required Values

4.1 Default Value with Colon Dash · Assign Default with Colon Equals

- The expansion ${var:-word} uses word when var is unset or empty.
- It does not assign the default back into the variable.
- This is useful for optional settings with a safe fallback.
- The expansion ${var:=word} assigns word when var is unset or empty.
- After the expansion, the variable keeps the default value.
- Use this when later commands should reuse the same fallback.

Examples

```
# Use default name
name=${USER:-student}
echo "$name"
# Default first argument
target=${1:-.}
ls "$target"
# Assign default port
: "${port:=8080}"
# Show assigned value
echo "$port"
```

4.2 Require Value with Colon Question · Alternate Value with Colon Plus

- The expansion ${var:?message} stops with an error when var is unset or empty.
- It is a compact way to require important inputs.
- The message should tell the user what value is missing.
- The expansion ${var:+word} uses word only when var is set and not empty.
- It is useful for adding optional flags or labels.
- If the variable is missing, the expansion becomes empty.

Examples

```
# Require a variable
: "${config:?config is required}"
# Require first argument
: "${1:?usage: script FILE}"
# Optional flag
debug=1
flag=${debug:+--verbose}
# Print alternate
name=Sam
echo "${name:+present}"
```

4.3 Defaults for Script Arguments · Empty Versus Unset in Defaults

- Positional parameters can use the same default expansion as named variables.
- The expression ${1:-.} means use the first argument or the current directory.
- Defaults make small scripts friendlier when an argument is optional.
- Colon forms treat empty and unset variables the same.
- Forms without the colon only test whether the variable is unset.
- Knowing the difference helps when an empty string is a meaningful value.

Examples

```
# Default directory
dir=${1:-.}
ls "$dir"
# Default name
name=${1:-student}
echo "$name"
# Colon treats empty as missing
x=""
echo "${x:-fallback}"
# No colon keeps empty
x=""
echo "${x-fallback}"
```

4.4 Quote Default Expansions

- Default expansions can produce values with spaces.
- Quoting the whole expansion keeps the chosen value as one argument.
- This is the same safe habit used with ordinary variables.

Examples

```
# Quoted default
name=${1:-Ada Lovelace}
printf "%s\n" "$name"
# Quoted path default
dir=${1:-my files}
ls "$dir"
```

5 Section 5

String Parameter Expansion

5.1 String Length · Substring Expansion

- The expansion ${#var} returns the length of a variable value.
- For simple ASCII text, the length is the number of characters.
- Length checks are useful for validation and simple reports.
- The expansion ${var:offset:length} extracts part of a value.
- Offsets start at zero for the first character.
- Substring expansion is useful for fixed-format names and dates.

Examples

```
# Measure a word
name=bash
echo "${#name}"
# Measure input
value="hello world"
printf "%s\n" "${#value}"
# First four characters
date=20260728
echo "${date:0:4}"
# Month characters
date=20260728
echo "${date:4:2}"
```

5.2 Replace First Match · Replace All Matches

- The expansion ${var/pattern/replacement} replaces the first matching part.
- The pattern uses shell pattern rules, not full regular expressions.
- It is convenient for small name changes without starting another command.
- The expansion ${var//pattern/replacement} replaces every matching part.
- It is useful when a repeated separator needs to change.
- Quote the expansion when the result is passed as an argument.

Examples

```
# Replace one dash
name=app-prod
echo "${name/-/_}"
# Replace extension
file=report.txt
echo "${file/.txt/.csv}"
# Replace all dashes
name=app-prod-east
echo "${name//-/_}"
# Replace spaces
title="daily report"
echo "${title// /_}"
```

5.3 Remove Shortest Suffix · Remove Longest Suffix

- The expansion ${var%pattern} removes the shortest matching suffix.
- It is often used to remove a file extension.
- The percent sign works from the end of the value.
- The expansion ${var%%pattern} removes the longest matching suffix.
- It is useful when a pattern can match more than one amount of text.
- Two percent signs make the suffix removal greedier.

Examples

```
# Remove extension
file=report.txt
echo "${file%.txt}"
# Remove after last dot
file=a.b.txt
echo "${file%.*}"
# Keep before first dot
file=a.b.txt
echo "${file%%.*}"
# Remove path tail pattern
path=/tmp/app/log.txt
echo "${path%%/*}"
```

5.4 Remove Prefix Patterns

- The expansion ${var#pattern} removes the shortest matching prefix.
- The expansion ${var##pattern} removes the longest matching prefix.
- Hash signs work from the beginning of the value.

Examples

```
# Remove shortest prefix
path=/tmp/app.log
echo "${path#*/}"
# Remove longest prefix
path=/tmp/app.log
echo "${path##*/}"
```

6 Section 6

Declarations and Variable Lifetime

6.1 declare · declare -i

- The declare command creates variables and can assign attributes.
- It is a Bash builtin, so it is mainly for Bash scripts.
- Declare helps document how a variable is intended to behave.
- The -i attribute asks Bash to treat assignments as arithmetic.
- This can be convenient for counters and simple numeric state.
- It can surprise beginners, so use it only when integer behavior is intended.

Examples

```
# Declare a variable
declare name="Sam"
# Show declared variables
declare -p name
# Declare integer
declare -i count=1
# Assign arithmetic
count=2+3
echo "$count"
```

6.2 declare -r · declare -a

- The -r attribute creates a readonly variable.
- Readonly variables cannot be reassigned or unset in the same shell.
- This is another way to express constants in Bash.
- The -a attribute declares an indexed array.
- Indexed arrays store multiple values under one variable name.
- Array indexes start at zero in Bash.

Examples

```
# Readonly with declare
declare -r app="demo"
# Inspect readonly value
declare -p app
# Declare array
declare -a names
# Assign array values
names=("Ann" "Bo")
```

6.3 declare -A · Shell Variables

- The -A attribute declares an associative array.
- Associative arrays use string keys instead of only numeric indexes.
- They are helpful for small lookup tables in Bash 4 and newer.
- A shell variable exists inside the current shell process.
- Child commands do not automatically receive ordinary shell variables.
- Use shell variables for temporary script state that outside programs do not need.

Examples

```
# Declare map
declare -A ports
# Assign by key
ports[web]=80
# Create shell variable
mode=dev
# Read in same shell
echo "$mode"
```

6.4 export - Send to Child Processes

- Exporting a variable places it in the environment for child processes.
- Programs started after export can read the exported value.
- Export only values that child programs actually need.

Examples

```
# Export a setting
export MODE=dev
# Child shell reads it
bash -c 'echo "$MODE"'
```

7 Section 7

Environment and PATH

7.1 Environment Versus Shell Variables · Inspect Environment Variables

- Environment variables are inherited by child processes.
- Shell variables stay in the current shell unless exported.
- Understanding the difference explains why some tools cannot see your value.
- The printenv command shows values that are present in the environment.
- It does not show every ordinary shell variable.
- Use it to confirm what child processes are likely to receive.

Examples

```
# Shell only
color=blue
bash -c 'echo "$color"'
# Exported value
export color=blue
bash -c 'echo "$color"'
# Print one value
printenv PATH
# Print all environment
printenv
```

7.2 Export While Assigning · Temporary Environment for One Command

- Bash can assign and export a variable in one command.
- This is common for configuration values used by tools.
- Keep secrets out of lecture examples and command history whenever possible.
- A variable assignment before a command can set environment only for that command.
- The current shell variable is not permanently changed by that temporary assignment.
- This pattern is useful for one-time settings.

Examples

```
# Assign and export
export APP_ENV=dev
# Use exported value
bash -c 'echo "$APP_ENV"'
# Set for one command
APP_ENV=test env | grep APP_ENV
# Set locale for date
LC_ALL=C date
```

7.3 Append to PATH Carefully · Prepend to PATH Carefully

- Appending adds a directory to the end of PATH.
- Commands in existing PATH directories keep priority over the appended directory.
- Quote the old PATH value so spaces or empty parts are preserved safely.
- Prepending adds a directory to the front of PATH.
- A prepended directory can override commands found later in PATH.
- Only prepend trusted directories because command search order changes.

Examples

```
# Append bin directory
PATH="$PATH:$HOME/bin"
# Export after append
export PATH="$PATH:$HOME/bin"
# Prepend local bin
PATH="$HOME/bin:$PATH"
# Check chosen command
command -v mytool
```

7.4 Remove Environment Values

- The unset command removes a variable from the shell and its future environment.
- Already-running child processes keep their own copies.
- Unset is useful when a temporary setting should no longer affect commands.

Examples

```
# Unset exported value
export MODE=dev
unset MODE
# Confirm missing value
printenv MODE
```

8 Section 8

Positional Parameters and Options

8.1 $0 and $1 · $#

- $0 usually contains the script name or the shell name.
- $1 contains the first positional argument passed to a script or function.
- Positional parameters are how simple scripts receive command-line input.
- $# expands to the number of positional arguments.
- It is useful for checking whether the user supplied enough input.
- Argument counts help produce clearer usage errors.

Examples

```
# Print script name
echo "script=$0"
# Print first argument
echo "first=$1"
# Print count
echo "count=$#"
# Require one argument
[ "$#" -ge 1 ] || exit 1
```

8.2 $@ · $*

- $@ represents all positional arguments.
- When quoted as "$@", each original argument stays separate.
- This is the safest way to forward arguments to another command.
- $* also represents all positional arguments.
- When quoted as "$*", the arguments join into one string.
- This behavior is different from quoted "$@" and is less often what scripts need.

Examples

```
# Forward arguments
printf "<%s>\n" "$@"
# Call another script
./worker.sh "$@"
# Join arguments
printf "<%s>\n" "$*"
# Compare with at
printf "<%s>\n" "$@"
```

8.3 Quoted $@ · shift

- Quoted "$@" preserves empty arguments and arguments with spaces.
- Each argument remains its own word after expansion.
- Use quoted "$@" when looping through or forwarding script arguments.
- The shift command removes the first positional argument.
- After shift, the old second argument becomes the new first argument.
- Shift is useful when a script handles options before the remaining values.

Examples

```
# Loop safely
for arg in "$@"; do
  echo "$arg"
# ...
# Forward safely
bash helper.sh "$@"
# Shift once
echo "$1"
shift
# ...
# Shift in a loop
while [ "$#" -gt 0 ]; do
  shift
# ...
```

8.4 getopts Intro

- The getopts builtin parses short options such as -v and -f file.
- It updates variables so your script can react to each option.
- Getopts is a good next step after simple positional arguments.

Examples

```
# Parse one flag
while getopts "v" opt; do
  echo "$opt"
# ...
# Parse option value
while getopts "f:" opt; do
  echo "$OPTARG"
# ...
```

9 Section 9

Reading Input and IFS

9.1 read -r · read -p

- The read command stores input from standard input into variables.
- The -r option prevents backslashes from being treated as escapes.
- Use read -r for most script input so text is preserved accurately.
- The -p option shows a prompt before reading input.
- It is convenient for small interactive scripts.
- Prompt text should make the expected input clear.

Examples

```
# Read one line
read -r line
# Print the line
printf "%s\n" "$line"
# Prompt for name
read -r -p "Name: " name
# Use the answer
echo "Hello, $name"
```

9.2 read Multiple Fields · IFS with read

- Read can assign separate input fields to multiple variables.
- IFS controls how the line is split into those fields.
- Extra text goes into the last variable when there are more fields than names.
- Setting IFS before read changes the separator for that read command.
- This is useful for colon-separated or comma-separated text.
- Keep the IFS assignment on the same command when possible.

Examples

```
# Read two fields
read -r first last
# Show fields
printf "%s,%s\n" "$first" "$last"
# Read colon fields
IFS=: read -r user shell <<< "sam:/bin/bash"
# Print parsed fields
printf "%s uses %s\n" "$user" "$shell"
```

9.3 Here-String · while read Loop

- A here-string sends a short string to a command's standard input.
- The syntax uses three less-than characters before the string.
- It is handy for examples and small parsing tasks.
- A while read loop processes input one line at a time.
- The read -r form preserves backslashes in each line.
- This pattern is common for reading files safely.

Examples

```
# Feed read
read -r word <<< "hello"
# Count characters
wc -c <<< "hello"
# Read a file
while read -r line; do
  echo "$line"
# ...
# Read command output
printf "a\nb\n" | while read -r x; do
  echo "$x"
# ...
```

9.4 Read Lines into an Array

- The readarray builtin reads lines into an indexed array.
- The -t option removes trailing newlines from each element.
- This is useful when you need to keep all lines for later processing.

Examples

```
# Read file lines
readarray -t lines < input.txt
# Print first line
printf "%s\n" "${lines[0]}"
```

10 Section 10

Arrays and Arithmetic

10.1 Indexed Arrays · Array Expansion

- An indexed array stores multiple values under one variable name.
- Indexes start at zero in Bash.
- Arrays are safer than packing multiple values into one space-separated string.
- The expansion "${array[@]}" produces each array element as a separate word.
- This preserves elements that contain spaces.
- Use this form when passing array values as command arguments.

Examples

```
# Create an array
names=("Ann" "Bo" "Cy")
# Read first item
echo "${names[0]}"
# Print each item
printf "<%s>\n" "${names[@]}"
# Pass to command
mkdir -p "${dirs[@]}"
```

10.2 Array Length · Associative Arrays

- The expansion ${#array[@]} returns the number of array elements.
- It counts elements, not characters.
- Length checks help decide whether an array has any values to process.
- An associative array uses text keys instead of numeric indexes.
- Bash 4 and newer support associative arrays.
- They are useful for small maps such as names to ports or codes.

Examples

```
# Count names
names=("Ann" "Bo")
echo "${#names[@]}"
# Check nonempty
[ "${#names[@]}" -gt 0 ] && echo "has names"
# Create a map
declare -A port=([web]=80 [ssh]=22)
# Read by key
echo "${port[web]}"
```

10.3 Arithmetic Expansion · Arithmetic Command

- The expression $((...)) performs integer arithmetic.
- Variables inside arithmetic expansion do not need a dollar sign.
- The result expands to text that can be printed or assigned.
- The command ((...)) evaluates integer arithmetic as a command.
- It is often used for increments and numeric tests.
- Its exit status depends on whether the arithmetic result is zero.

Examples

```
# Add numbers
echo "$((2 + 3))"
# Increment value
count=1
count=$((count + 1))
# Increment counter
count=0
((count++))
# Numeric test
count=3
((count > 0)) && echo "yes"
```

10.4 let - Arithmetic Builtin

- The let builtin evaluates arithmetic expressions.
- It is older and less visually clear than $(( )) or (( )).
- You may see let in existing scripts, so it is useful to recognize.

Examples

```
# Add with let
count=1
let "count=count+1"
# Print result
echo "$count"
```

11 Section 11

Substitution and Special Variables

11.1 Command Substitution · Backticks Versus Dollar Parentheses

- Command substitution runs a command and inserts its output.
- The modern form uses dollar sign and parentheses.
- Quote command substitutions when the output should stay one argument.
- Backticks are the older form of command substitution.
- The dollar-parentheses form is easier to nest and read.
- Prefer the modern form in new Bash scripts.

Examples

```
# Capture date
today=$(date +%Y-%m-%d)
# Print captured value
echo "$today"
# Old form
today=`date +%Y-%m-%d`
# Modern form
today=$(date +%Y-%m-%d)
```

11.2 Command Substitution Newlines · Process Substitution Intro

- Command substitution removes trailing newline characters from command output.
- Internal newlines can remain inside the captured value.
- Be careful when capturing output that is meant to preserve exact file content.
- Process substitution lets a command read the output of another command as if it were a file.
- Bash commonly uses the forms <(command) and >(command).
- It is useful with tools that expect filenames instead of pipelines.

Examples

```
# Capture output
files=$(printf "a\nb\n")
# Show captured text
printf "[%s]\n" "$files"
# Compare two streams
diff <(sort a.txt) <(sort b.txt)
# Read generated input
while read -r x; do echo "$x"; done < <(printf "a\n")
```

11.3 $? · $$

- $? expands to the exit status of the most recent foreground command.
- Zero usually means success, and nonzero usually means failure.
- Read it immediately because the next command replaces the value.
- $$ expands to the process ID of the current shell.
- A process ID is a number the operating system uses to identify a running process.
- It can be useful in temporary names, but safer tools also exist for temp files.

Examples

```
# Show status
false
echo "$?"
# Check after command
ls missing.txt
echo "$?"
# Print shell PID
echo "$$"
# Use in a name
tmp="run-$$.log"
echo "$tmp"
```

11.4 $! and $_

- $! expands to the process ID of the most recent background command.
- $_ often expands to the last argument of the previous simple command.
- These variables are convenient, but clear named variables are easier for beginners.

Examples

```
# Capture background PID
sleep 5 &
echo "$!"
# Show last argument
mkdir -p logs
echo "$_"
```

12

Section 12

Formatting, Comparisons, and Review

12.1 printf Formatting · printf Versus echo -e

- Printf uses placeholders to control how values are displayed.
- The %s placeholder prints a string value.
- Format strings make output predictable for logs and generated files.
- Echo behavior for escape sequences can vary between shells and systems.
- Printf handles escapes through its format string in a predictable way.
- Use printf when newlines, tabs, or exact output matter.

Examples

```
# Print two strings
printf "Name: %s\n" "$name"
# Print table row
printf "%-10s %s\n" "$user" "$role"
# Portable newline
printf "one\ntwo\n"
# Avoid relying on echo -e
echo -e "one\ntwo"
```

12.2 Boolean Strings · String Compare Preview

- Bash does not have a separate Boolean variable type for ordinary strings.
- Scripts often use values such as true, false, yes, no, 1, or 0.
- Choose one convention and compare it explicitly.
- String comparisons test text values.
- The single equal sign is commonly used inside the test command for string equality.
- Always quote variables in tests so empty values do not break the command shape.

Examples

```
# Store a flag
enabled=true
# Check a flag
[ "$enabled" = true ] && echo "on"
# Compare strings
[ "$mode" = "dev" ] && echo "dev mode"
# Compare nonempty
[ -n "$name" ] && echo "$name"
```

12.3 Numeric Compare Preview · Common Quoting Bugs

- Numeric comparisons treat values as numbers instead of text.
- Arithmetic commands are a clear Bash-style way to compare integers.
- Do not use string ordering when you mean numeric ordering.
- Missing quotes can split file names that contain spaces.
- Missing quotes can expand wildcard characters from data into filenames.
- A reliable first fix is to quote variable expansions unless you need splitting.

Examples

```
# Compare integers
count=3
((count > 1)) && echo "many"
# Add before compare
total=$((a + b))
((total >= 10))
# Buggy path use
file="my notes.txt"
cat $file
# Safer path use
file="my notes.txt"
cat "$file"
```

12.4 Checkpoint - Variables and Quoting

- You should be able to assign variables without spaces and read them with expansion.
- You should know when double quotes, single quotes, and braces change behavior.
- You should practice safe patterns with arguments, defaults, arrays, and command substitution.

Examples

```
# Safe variable use
name="Ada Lovelace"
printf "%s\n" "$name"
# Safe argument loop
for arg in "$@"; do
  printf "%s\n" "$arg"
# ...
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory
02 · Variables